introduction to shared memory architectures and models

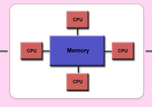
abstraction vs hardware

- it's crucial to distinguish between three layers of parallel computing
  - **parallel patterns** which describe the algorithm itself
    - parallel patterns = logica pura del problema
  - **parallel architectures** which represent the physical hardare, memories and interconnections
    - chip fisici, la RAM, i bus
  - **parallel programming model** is an abstraction layer above the hardware
    - regole e strumenti che usiamo per scrivere il codice
- **POSIX Threads** and **OpenMP** are specific implementations of a shared memory parallel programming model based on threads
  - they are not just libraries
- key takeaway a single programming model
  - can have multiple implementations (like Pthreads or OpenMP)
  - does not always map one-to-one to the underlying hardware

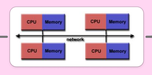
hardware architectures: UMA vs NUMA

when we look at the physical hardware, we generally categorize architectures based on how memory is accessed

- **shared memory**



  - multiple CPUs all connected via a bus to a single, large memory space
  - this is often rederred as to **UMA (Uniform Memory Access)** because every processor accesses the memory at the same speed
    - one place where we put everything
- **distributed memory**



  - every CPUs has its own local memory and they are connected via a network
  - this is often a **NUMA (Non-Uniform Memory Access)** system
    - accessing your local memory is fast, but accessing another CPU's memory requires going through the network, which introduces latency
    - while powerful, this is more complex to manage structurally

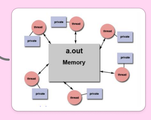
thread model

programming models

- we use **programming models** to abstract the hardware
  - **shared memory (without threads)**
    - using processes
  - **threads**
  - **message passing**
    - like MPI, used often for distributed memory
  - **data parallel**
    - focusing on distributing data rather than tasks
  - **Single Program Multiple Data**
  - **Multiple Program Multiple Data**
- the model is an abstraction
  - you could theoretically run a "Shared Memory" model on "Distributed Hardware" via software (Visual Shared Memory), though performance would depend on the physical constraints
- **the shared memory model (process-based)**
  - before jumping into threads, we look at the simples shared memory model using **processes**
    - in this model, multiple processes share a common physical address space
      - in pratica, se il processo A scrive all'indirizzo virtuale 0×1000 mentre il processo B legge all'indirizzo virtuale 0×5000, il sistema operativo sa che entrambi quei numeri puntano alla stessa cella fisica (che è in comune)
    - however, the Operating System maps this to different virtual address spaces for each process

> La frase sottolinea che, sebbene la memoria sia fisicamente condivisa, l'OS deve comunque integrarla nell'illusione di isolamento ("Virtual Address Space") di ciascun processo.
>
> Ecco perché è importante:
>
> 1. **Protezione:** Il resto della memoria rimane privato. Solo quel pezzettino è mappato in comune.
> 2. **Flessibilità:** Il Processo A potrebbe avere lo spazio di indirizzi `0x1000` occupato da altro codice, quindi l'OS mappa la memoria condivisa in un punto libero (`0x2000`). Il Processo B potrebbe averlo libero a `0x1000` e mapparlo lì.
>
> **In sintesi:** La frase non vuol dire che i dati vengono separati o copiati (sarebbe inutile). Vuol dire che **lo stesso dato fisico è accessibile tramite "nomi" (indirizzi virtuali) che appartengono agli spazi privati dei due processi.** È come avere due chiavi diverse che aprono la stessa porta.

  - pros
    - no explicit communication required
    - you don't need to "send" data but just read and write to shared variables
  - cons
    - difficult to maintain data locality (keeping data close to the CPU)
    - processes are "heavy"
      - **context switching** - stopping one process to start another - is computationally expensive for the Operating System
        - poiché è impegnativo creare un processo, il cambio di contesto tra i processi è lento
- **introduction**
  - used by POSIX Threads and OpenMP
  - in this model, we have a single process containing multiple threads that execute concurrently
    - this is a "lighter" approach compared to multi-processing



  - memory is structured in this way
    - **shared resources**
      - all threads can access the resources acquired by the main process (global variables, files, code)
    - **private resources**
      - each thread also has its own **local data** (private stack/registers)
  - this model mimics the theoretical **PRAM (Parallel Random Access Machine)** model
    - we have concurrent execution streams (T1, T2, T3,...) over time, all working on a shared state but capable of independent computation
    - this combination of shared global access and private local data is what makes multithreading powerful
      - also introduces challenges like data races

> Il **Thread Model** (usato da OpenMP/Pthreads) è l'implementazione concreta del modello teorico PRAM (Shared Memory). In particolare, si comporta come una macchina CRCW (Concurrent Read, Concurrent Write) dove però la gestione delle scritture concorrenti ("Concurrent Write") è demandata interamente alla sincronizzazione manuale del programmatore per evitare risultati corrotti.

  - to understand thread we must first look at their container: the **UNIX Process**
    - unità di allocazione delle risorse
    - a process is a comprehensive execution environment created by the operating system
      - un processo viene creato dal sistema operativo e richiede una quantità significativa di "overhead" (risorse e tempo) per essere inizializzato
    - it contains everything needed to run a program
      - the program instructions (Text), resources like Process ID; User ID, environment variables and the working directory
    - it has its own distinct memory address space, divided into the Stack, Text, Data and Heap

> **2. Da cosa è composto (La "Lista della Spesa")**
>
> Le slide elencano precisamente cosa contiene un processo UNIX. Immaginalo come un pacchetto che include:
>
> - **Identificativi:** Process ID (PID), ID utente, ID gruppo.
> - **Contesto di Esecuzione:**
>   - **Istruzioni:** Il codice del programma vero e proprio.
>   - **Registri:** Lo stato attuale del processore.
>   - **Stack:** La memoria per le chiamate di funzione e variabili locali.
>   - **Heap:** La memoria dinamica globale.
> - **Risorse di Sistema:**
>   - **File Descriptors:** I file aperti.
>   - **Environment:** Variabili d'ambiente e directory di lavoro.
>   - **Librerie condivise** e strumenti di IPC (Inter-Process Communication).

  - a **thread**, by definition, is an "independent stream of instructions" running *within* that process
    - unità di esecuzione delle risorse
    - we can think of it as any procedure that run independently from the main processes
      - thread can be scheduled to run by the operating system
      - any thread can execute at the same time as other threads
      - threads can be created *dynamically* during execution
    - **example** of multi-thread program
      - main which calls some procedures
      - all the procedure run *simultaneously and/or indpendently* by the operating system
    - thread are **lighter** than processes
      - while a process duplicated all resources, a thread duplicates only the bare essential needed to run independently
        - ogni thread ha la usa area di stack riservata all'interno dello spazio di indirizzi del processo → non rappresenta una copia ma è solo una partizione
      - each thread maintains its own private resources
        - **stack pointer and stack** to track its own function calls and local variables
        - **program counter** to know which instruction it is executing
        - **registers** to hold its current working values

> Nel **Thread Model**, il Processo UNIX diventa l'**ambiente condiviso**.
>
> - Invece di avere un processo per ogni compito (che dupliche rebbe tutto l'elenco qui sopra per ogni task), usiamo un **singolo processo** per ospitare **molti thread**.
> - **Cosa viene condiviso?** Tutti i thread dentro quel processo condividono le **istruzioni**, l'**heap** (memoria globale), i **file aperti** e l'**ambiente**.
> - **Cosa rimane privato?** Ogni thread mantiene solo il proprio **Stack** (per non mischiare le chiamate di funzione) e i propri **Registri** (per sapere a che punto è arrivato).

processes and threads

- while they maintain private execution contexts, thread share the vast majority of the process's resources
  - they all inhabit the same **User Address Space**
  - this means they share
    - **code (text):** they run the same program functions
    - **data and heap:** they access the same global variables and dynamic memory (like arrayA or arrayB)
    - **OS resources:** they share open files, locks and sockets
- changers made by one thread to shared system resources (such as closing a file) will be seen by all other threads
  - two pointers having the same value point to the same data
- this architecture allows for **implicit communication**
  - if Thread 1 updates a global variable or closes a file, Thread 2 sees that change immediately because they are looking at the exact same memory address
  - you don't need complex message-passing protocols but you just read and write to shared variables
  - reading and writing to the same memory locations requires **explicit synchronization** by the programmer

> "In generale, il Thread Model rappresenta l'implementazione concreta del modello teorico PRAM CRCW (con risorse limitate). Si basa su flussi di istruzioni (stream) eseguiti in parallelo che permettono Concurrent Read/Write; le scritture concorrenti, tuttavia, richiedono sincronizzazione esplicita (lock/mutex) da parte dell'utente per evitare race conditions. Architetturalmente, si fonda sullo UNIX Process come unità di allocazione risorse (il "contenitore" pesante). All'interno di esso, vivono i Thread, unità di esecuzione (lightweight) schedulate dal Sistema Operativo.
> I thread condividono lo spazio di indirizzi globale (Codice, Dati, Heap, File Descriptor) permettendo una comunicazione implicita rapidissima, ma mantengono private le risorse necessarie all'esecuzione indipendente: Program Counter, Registri e Stack. Questa condivisione, pur efficiente, espone al rischio di interferenze (Race Conditions) e problemi di coerenza della cache (False Sharing)."

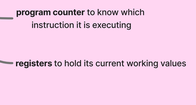
- **power of shared resources**
  - this ease of sharing brings the significant danger of **data race**
    - since threads execute concurrently - potentially at the exact same time on different cores - we run into trouble when they try to touch the same data without coordination
  - **data races**
    - a data race occurs specifically when three conditions are met
      - two thread (or processors) access the **same variable**
      - at least **one** of them performs a **write** operation
      - these accesses are **concurrent**
    - **example**
      - consider this case with s a shared global variable
        - Thread 1 executes s = s + f(i)
        - Thread 2 executes s = s + g(i)
      - both threads might read the old value of s simultaneously, calculate their addition locally and then write back
        - one of the update will be lost
        - this creates an inconsistent state
    - therefore, while reading and writing shared memory is efficient, it requires **explicit synchronization** by the programmer to be safe
      - we must strictly control who accesses shared data and when

> **A. Mutua Esclusione (Synchronization)**
>
> È il metodo più comune nel modello Shared Memory (OpenMP/Pthreads).
>
> - **Come funziona:** Usi un **Lock** (o Mutex, o Critical Section).
> - **Logica:** "Solo un thread alla volta può toccare questa variabile".
> - **Pro:** Facile da ragionare.
> - **Contro:** Rallenta l'esecuzione (serializzazione) e può causare Deadlock (blocco totale).
>
> **B. Thread-Local Storage (Isolamento)**
>
> Si basa sull'usare le risorse private (Stack) invece di quelle condivise.
>
> - **Come funziona:** Invece di usare una variabile globale, la funzione lavora solo su variabili locali passate come argomenti o allocate nello stack del thread.
> - **Logica:** "Ognuno ha la sua copia, così non litighiamo".
> - **Esempio:** Usare `strtok_r` dove sei tu a fornire il buffer di memoria, invece di far usare alla funzione il suo buffer statico condiviso.
>
> **C. Operazioni Atomiche**
>
> Si usa l'hardware per fare operazioni indivisibili.
>
> - **Come funziona:** Usi istruzioni speciali della CPU (es. *Compare-and-Swap*).
> - **Logica:** "Aggiorno questo numero così velocemente che nessuno può inserirsi nel mezzo".
> - **Nota:** È molto veloce ma difficile da implementare per strutture dati complesse.

designing threaded programs

- the first step to design a safe and efficient multithreaded program is architectural
  - *can my program be organized into discrete, indipendent tasks?*

    - we aren't just running routine1 follows by routine2 but we are looking for opportunities where routine1 and routine2 can overlap in time
  - simply finding independent tasks isn't enough but we have to ensure that the code within those tasks is **Thread-Safe**
- **concept of thread safety**
  - a function is "thread-safe" if it is called simultaneously by multiple threads without causing data corruption
    - un pezzo di codice (una funzione, una libreria o un oggetto) si dice thread-safe se può essere invocato da più thread contemporaneamente senza rompere nulla (senza produrre risultati errati, crash o race conditions)
    - consider a standard mathematical library or a random number generator
      - often these legacy functions use static variables internally to keep track of state
    - if Thread 1, Thread 2 and Thread 3 all call a function subA that modifies a specific memory location (like 0×4450A) in our example) at the same time we have a collision



      - the internal state gets corrupted
      - therefore, when designing threaded programs, you have a responsability
        - we must ensure that any shared data is protected by synchronization primitives
        - we must design our tasks to use private local data as much as possible to avoid these conflicts entirely

common parallel programming models

- once we have identifies safe tasks, we need a structure to manage them
  - **manager/worker model**
    - we have a single "manager" that breaks the work down into chunks among them
    - it then spawns a set of "worker" threads and distributes these chunks among them
      - the manager typically waits for the workers to finish and then aggregates the results
    - we start with a main Task 1 (the manager). It identifies sub-problems—Task 2, Task 3, Task 4 —and assigns them to be executed in parallel

      - abbiamo due tipologie di thread:
        - manager → il suo compito non è fare i calcoli pesanti ma generare i task, distribuirli e raccogliere i risultati finali
        - workers → prendono un task dal manager, lo eseguono, restituiscono il risultato e ne chiedono subito un altro
      - il bottleneck è il manager → se i worker sono troppo veloci e il manager è lento a preparare o distribuire i lavori, i worker restano in attesa
  - **pipeline model**
    - data flows through a series of stages

      - Thread A might do the first step of processing
      - pass the buffer to Thread B
      - which passes it to Thread C
      - each thread work continuously on different pieces of data at different stages of completion
    - il problema viene diviso in una sequenza di stadi dove ogni stadio esegue un'operazione specifica sui dati e passa il risultato allo stadio successivo
    - la pipeline non riduce il tempo per fare una singola latenza ma aumenta la quantità di unità prodotte per ora

how do we implement threads?

- historically, implement threads was quite difficult
  - hardware vendors used to provide their own proprietary versions of threading libraries
    - this means code written for one machine wouldn't necessarily work on another
    - to solve this portability nightmare, the industry moved towards standardization
  - today, the programmer is responsible for explicitly handling parallelism and synchronization but they do so through standardized interfaces
    - there are two main approaches we will study
      - **Pthreads approach**
        - you make explicit function calls to the operating system to manage threads
      - **OpenMP approach**
        - you give hints to the compiler using annotations and it generates the threading code for you

> POSIX threads and OpenMP are two implementations of a **shared memory** *parallel programming model* with **threads**